Rhys Wiers
CIS 343
Dr. Haoyu Li
October 19, 2026

Lab 3: Parser

1. **Grammar:**

expression	→	logic_or
logic_or	→	logic_and (`"||"` logic_and)∗
logic_and	→	equality (`"&&"` equality)∗
equality	→	comparison ((`"!="` | `"=="`) comparison)∗
comparison	→	term ((`">"` | `">="` | `"<"` | `"<="`) term)∗
term	→	factor ((`"-"` | `"+"`) factor)∗
factor	→	unary ((`"/"` | `"*"` | `"%"`) unary)∗
unary	→	(`"!"` | `"-"`) unary | primary
primary	→	`NUMBER` | `STRING` | `"true"` | `"false"` | `"nil"` | `"("` expression `")"`

2. **Associativity** All grammar rules are left associativy with the exception of the unary operators.

3. **Test Cases** From the project root, you can run the following to compile and run the parser tests:

```
g++ -std=c++17 -Isrc test/lab3/parser-test.cpp $(ls src/*.cpp | \
grep -v main.cpp) -o parser-test
```

Then run the executable:

```
./parser-test
```

The following is the output I got of the test cases. It covers all grammar rules that the parser handles.

```
ok   number
ok   decimal
ok   string
ok   true
ok   false
ok   nil
ok   unary minus
ok   unary bang
ok   nested bang
ok   nested minus
ok   group
ok   nested group
ok   factor over term
ok   term after factor
ok   group overrides
ok   unary over factor
ok   unary over equality
ok   term over comparison
ok   comparison over equality
ok   and over or
ok   and before or
ok   remainder is factor
ok   assignment example
ok   minus
ok   slash
ok   equality
ok   or
ok   binary then unary
ok   full
ok   logic
[line 1] Error at end: Expected expression.
ok   empty input
[line 1] Error at end: Expected expression.
ok   trailing operator
[line 1] Error at '*': Expected expression.
ok   leading operator
[line 1] Error at '+': Expected expression.
ok   double operator
[line 1] Error at end: Expected ')' after expression.
ok   missing close paren
```

```
[line 1] Error at ')': Expected expression.
ok   empty group
[line 1] Error at ')': Expected expression.
ok   lone close paren
[line 1] Error at '2': Expected end of expression.
ok   trailing token

All tests passed.
```

The errors are caused by:

(a) Empty input

(b) "+" with nothing after

(c) Two "+" in a row

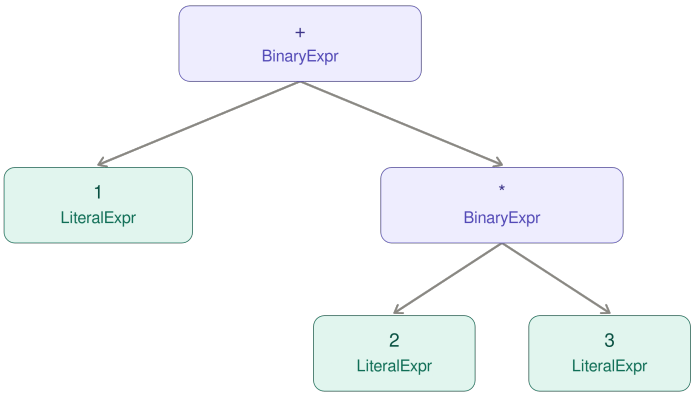
(d) No closing parenthesis after an opening one earlier in the expression

(e) Empty parentheses

(f) Two numbers with nothing in between

4. **Example AST**

   Here is the tree for "1 + 2 * 3":



   The parser first reads 1 and holds onto it. Then it sees +, so it knows it needs something to add to 1, but before deciding what, it looks at what comes next. It reads 2 and then sees *. Multiplication has higher precedence than addition, so the 2 belongs with the 3, not with the 1. It bundles 2 * 3 together. Now it can finish the addition: 1 + (2 * 3).

   This results in the tree shown above.